

Expense Split, Budget Encumbrance & Approval Engine

Complete phase-by-phase record of the work that lets one expense line item be split across several Cost Centers, reserves budget the moment a report is submitted instead of when it's paid, and routes each Cost Center's own owner into the approval chain — built across Phases 0–8 on the `sindhu` branch.

9	46	3	5	875
PHASES (0–8)	FILES TOUCHED	NEW ENTITIES	NEW ENDPOINTS	TESTS PASSING

00 Existing Architecture Mapping

ANALYSIS ONLY

WHY THIS PHASE EXISTED

Before touching a live approval/budget engine, we mapped exactly how it worked today — so every later phase could add behavior without breaking normal, non-split expense reports.

WHAT WAS CONFIRMED, UNCHANGED

ExpenseReport.costCenter stays non-nullable — every report still has exactly one header Cost Center

CostAllocation the legacy allocation entity is left in place, untouched, pending a separate frontend-cutover decision

ApprovalWorkflowServiceImpl / **DefaultApproverSourceResolverImpl** / **ChainCorrectnessServiceImpl** / **MaterialChangeEvaluator** / **ApPaymentServiceImpl**

fenced off from Phase 1 — each was opened only in the phase that specifically needed it (4, 4, 4/8, untouched, 6)

Output: an 8-point correction list plus an explicit "do not touch yet" boundary, both re-confirmed before any code was written.

01 Data Foundations

BUSINESS OUTCOME

Introduces the data model for "split one expense across multiple Cost Centers" and "reserve budget early" — with zero effect on any existing, unsplit report.

NEW ENTITIES

ExpenseSplit NEW

splitId PK, UUID

lineItem, costCenter FK, both NOT NULL

splitType PERCENTAGE | FIXED_AMOUNT, snapshotted at creation

percentage decimal(7,2), only for PERCENTAGE mode

allocatedAmount decimal(19,4) NOT NULL — resolved monetary value in Org Base Currency

splitOrder int NOT NULL — deterministic rounding-remainder target

unique (lineItemId, costCenterId)

ApprovalSplitReview NEW

reviewId PK, UUID

split, assignment FK — keyed by (split, assignment), not (split, level) — this is what lets one combined owner cover several splits

status reuses LineItemReviewStatus: PENDING / APPROVED / NEEDS_CORRECTION

comment, actedBy, actionedAt, version same shape as ApprovalLineItemReview

unique (splitId, assignmentId)

BudgetEncumbrance NEW

encumbranceId PK, UUID

budget FK, nullable — null means unbudgeted spending

report FK NOT NULL

split FK, nullable — null represents the report's "unsplit remainder"

submissionCycle int NOT NULL

amount decimal(19,4) NOT NULL, Org Base Currency

status ACTIVE → RELEASED or → CONSUMED

unbudgeted boolean, default false

createdAt, releasedAt, consumedAt, version full lifecycle timestamps + optimistic lock

NEW ENUMS

SplitType PERCENTAGE, FIXED_AMOUNT

BudgetEncumbranceStatus

ACTIVE, RELEASED, CONSUMED

MODIFIED ENTITIES

ExpenseLineItem

+ expenseSplits **EXTENDED**

ApprovalAssignment

+ splitReviews **EXTENDED**

CostCenter + allowUnbudgeted (boolean, NOT NULL)

CostCenterBudget

+ rolloverFromPrevious, allowRollover, rolloverCap, warningThreshold

MIGRATION

V15__budget_policy_flags_backfill.sql — backfills allow_rollover / allow_unbudgeted to FALSE on existing rows, then locks both NOT NULL. Every other new table/column is left to hibernate.ddl - auto=update, matching this project's existing convention.

NEW REPOSITORIES

ExpenseSplitRepository,
ApprovalSplitReviewRepository,
BudgetEncumbranceRepository (initial shape).

02

Split Validation & Management

BUSINESS OUTCOME

An employee can split one expense line item across 2 or more Cost Centers, by percentage or by fixed amount — with the system guaranteeing the split always adds up exactly to the line item total, and never silently merging or guessing at ambiguous input.

LOCKED RULES

Rule 1 — a split set is 0 rows (normal expense) or 2+; exactly 1 is always rejected.

Rule 2 — the same Cost Center twice on one line item is rejected outright, never auto-merged.

Rule 3 — FIXED_AMOUNT splits must sum to the line item total within ± 0.01 , no rounding absorption.

Rule 4 — PERCENTAGE splits must sum to 100% within $\pm 0.01\%$.

Rule 5 — for PERCENTAGE mode, the rounding remainder is absorbed entirely into the *last* split in entry order, so the stored sum reconciles exactly.

All splits on one line item share one mode — PERCENTAGE and FIXED_AMOUNT can never mix within the same line item.

NEW DTOS

ExpenseSplitRequest

costCenterId, splitType, percentage, allocatedAmount

ExpenseSplitReplaceRequest

List<ExpenseSplitRequest> — whole-set replace, not per-row CRUD

ExpenseSplitResponse

splitId, lineItemId, costCenterId/Name, splitType, percentage, allocatedAmount, splitOrder, createdAt

ENDPOINTS

GET /xms/employee/expense-line-items/{lineItemId}/splits
list the current splits

PUT /xms/employee/expense-line-items/{lineItemId}/splits
replace the whole set; roles
ADMIN/GENERAL/FINANCE/MANAGER, ownership
enforced in the service

Authorization is deliberately stricter than the legacy CostAllocationService it sits alongside: every call requires the acting employee to own the report, and the report must be in an editable status.

16 tests ExpenseSplitServiceImplTest — every rule above, plus the Phase 6 currency-basis fix

03 Budget Encumbrance Engine

BUSINESS OUTCOME

Budget is now reserved the moment a report is *submitted*, not when it's finally paid — so two people can no longer both spend the same last dollar of a Cost Center's budget while their reports are mid-approval. Admins also get three new budget controls: carrying forward unused budget from a prior year (capped), a low-budget warning, and an explicit "allow spending with no budget configured" override.

EFFECTIVE AVAILABLE BUDGET

`availableBudget` – `SUM(amount)` across every `ACTIVE` `BudgetEncumbrance` against that budget row

Never stored — computed at read time, every time. `CostCenterBudget.availableBudget` itself is never touched by encumbrance logic; only the existing, unmodified `consumeBudget()` ever changes it, and only at AP payment (Phase 6).

BUDGETENCUMBRANCESERVICE (NEW)

`validateAndEncumber(report, cycle)`

atomic all-or-nothing; locks every distinct Cost Center's budget row in sorted UUID order (deadlock-safe); same Cost Center across several splits is checked once, combined

`releaseActiveForCycle(reportId, cycle)`

`ACTIVE` → `RELEASED`; no-op if none active

`consumeActiveForReport(reportId, cycle)`

`ACTIVE` → `CONSUMED` via the existing `consumeBudget`; returns whether anything was found

`effectiveAvailable(budget)`

the read-time formula above

NEW BUDGET-POLICY FIELDS

`CostCenterBudget.rolloverFromPrevious`

validated ≤ `MIN(source's true unencumbered remainder, rolloverCap)`

`allowRollover, rolloverCap, warningThreshold`

admin-set per budget row

`CostCenterBudgetRequest.rolloverSourceBudgetId`

request-only, never persisted — `fiscalYear` is free text, so "last year's row" can't be auto-derived safely

`CostCenter.allowUnbudgeted`

lets a Cost Center accept encumbrances with zero budget configured

First pessimistic lock (`PESSIMISTIC_WRITE`) ever introduced in this codebase — used only by `validateAndEncumber`, never by `consumeBudget`.

Built and fully tested standalone in this phase — deliberately **not yet** called from submission or AP completion, so the capability could be verified in isolation first (wired in Phases 5 & 6).

22 tests `CostCenterBudgetServiceImpl`

15 tests `CostCenterBudgetController`

19 tests `BudgetEncumbranceServiceImpl`

04 Approval Routing for Splits

BUSINESS OUTCOME

When an expense is split across Cost Centers, each Cost Center's own owner now approves their own portion — not one person rubber-stamping the whole report. If the same person owns two of the split Cost Centers, they get *one* combined task, not two. A report that's part split, part normal still needs both the split owners *and* the original approver to sign off.

LOCKED DESIGN DECISIONS

Split-owner assignments always activate **immediately, in parallel** — the level's SEQUENTIAL/ANY_OF/ALL_OF quorum only governs its normal (non-split) entries, since each owner approves a disjoint slice with nothing to sequence.

Scoped to APPROVAL-type levels only; Finance Verification is untouched this phase.

A split-owner assignment auto-skipped by the existing duplicate-approver pass has its splits **waived** from level completion — the same reasoning as why that skip exists at all.

CORE MECHANICS

createSplitOwnerAssignments

resolves one owner per distinct split Cost Center; same-owner splits combine into one ApprovalAssignment

isEntireInstanceDone

= normal line-item track done AND every non-skipped split-owner's own splits approved

reviewSplit(reportId, splitId, actor, decision)

the split-aware sibling of reviewLineItem — one owner's rejection never blocks a different owner

ENDPOINT

POST /xms/approvals/{reportId}/splits/{splitId}/review
approve or flag one split's Cost Center Owner responsibility

Zero changes needed to ChainCorrectnessServiceImpl or DefaultApproverSourceResolverImpl — both already operate generically on ApprovalAssignment rows, so self-approval redirect and duplicate-skip work on split-owner assignments for free (proven for real in Phase 8).

+10 tests ApprovalWorkflowServiceImplTest (41 total in file)

05

Correction & Encumbrance Lifecycle

BUSINESS OUTCOME

Reserved budget doesn't sit "phantom-locked" forever — the moment a report is cancelled, recalled, rejected, or fully restarted after correction, its budget reservation is released, and a fresh one is made only if the report is actually resubmitted.

WHERE ENCUMBRANCES GET CREATED

submit() `validateAndEncumber` runs before the approval chain is even materialized — insufficient budget aborts with nothing created

fullRestart() releases the old cycle first, then encumbers the new one — so the same money is never double-counted against itself

WHERE ENCUMBRANCES GET RELEASED

cancelAllOpenInstances()
the single existing choke point already used by recall, cancel, rejectReport, and full-restart's old-cycle teardown — one change covers all four

resumeInPlace()
deliberately untouched — a same-cycle Manager correction never re-validates the encumbrance amount (documented, accepted gap)

+7 tests

encumber-on-submit · insufficient-budget-abort · release-on-recall/cancel/reject · resume-in-place untouched · restart ordering (release-then-encumber, verified in sequence)

06

AP Payment Completion

BUSINESS OUTCOME

When Accounts Payable marks a report paid, the org's real budget is decremented correctly per Cost Center a split touched — not as one lump sum against the report's header Cost Center — and, for a foreign-currency expense, in the correctly converted amount.

BUG FOUND & FIXED — CURRENCY BASIS

`ExpenseSplitServiceImpl` (Phase 2) was splitting `lineItem.amount` — the line item's own display currency — instead of `lineItem.baseAmount`, the Org Base Currency figure that `CostCenterBudget` is actually denominated in. `BudgetEncumbranceServiceImpl`'s unsplit-remainder sum had the same issue. Invisible for a base-currency line item (the two are equal there); real for any foreign-currency one. Fixed in

both classes, with your explicit sign-off to reopen two already-confirmed phases; 2 new tests prove it with a foreign-currency fixture (amount = 10000, baseAmount = 8500).

BudgetEncumbranceService.consumeActiveForReport now returns boolean — found-and-consumed vs. nothing-to-consume

ApprovalWorkflowService.getCurrentSubmissionCycle(reportId)

new — lets AP ask "which cycle" without depending on the approval engine's own repository

ApPaymentServiceImpl.markPaymentCompleted tries encumbrance-based consumption first; falls back to the original direct consumeBudget(costCenter, fiscalYear, report.totalAmount()) call only when a report has zero encumbrances at all — covering reports already in flight before this wiring existed, with no double-consumption possible

+2 tests

currency-basis fix

+2 tests

ApPaymentServiceImplTest (19 total in file)

07 Queue, History & Bulk Approve

BUSINESS OUTCOME

A Cost Center Owner now sees their own pending splits in "My Approvals," can see the full review history of every split on a report, and can bulk-approve everything they personally have pending in one action — without ever touching a different owner's work.

BUG FOUND & FIXED — WRONG ASSIGNMENT MARKED COMPLETE

reviewLineItem's authorization lookup didn't exclude split-owner assignments. A split-owner who also happened to be resolvable for the shared line-item review would have their *own* split-owner assignment wrongly marked COMPLETED — even though their actual splits were still untouched. Fixed by excluding assignments with any split reviews from that lookup; surfaced by the very bulk-approve scenario this phase added.

NEW DTOS / FIELDS

PendingSplitResponse

splitId, reviewId, lineItemId,
costCenterId/Code/Name, allocatedAmount

SplitReviewResponse

ENDPOINT

GET

/xms/approvals/{reportId}/split-
reviews

split-aware sibling of the existing line-item-reviews
endpoint

BULKAPPROVE — REWRITTEN

current-cycle status/comment/actedBy per split, plus level context

ApprovalQueueItemResponse.pendingSplits

new field — scoped per-assignment, not level-wide like pendingLineItems

+4 tests

queue isolation between owners · split-review history · bulk-approve scoping (53 total across both files)

Now determines per caller whether they hold a normal-track assignment, split-owner assignment(s), or both — and only attempts the track(s) they actually have.

08 Cross-Cutting Integration Testing

BUSINESS OUTCOME

Proof the whole system works together, end to end — not just each piece in isolation. This is exactly the class of testing that caught the two bugs found in Phases 6 and 7.

New test class SplitApprovalBudgetIntegrationTest wires the **real** ApprovalWorkflowServiceImpl + ChainCorrectnessServiceImpl + BudgetEncumbranceServiceImpl together (repositories mocked, no database) — a new pattern for this codebase, where every other test mocks out the other collaborators.

SCENARIO	PROVES
Full split lifecycle	submit → both owners + header approve independently → APPROVED → encumbrances stay ACTIVE
Self-approval redirect	submitter owns a split's Cost Center → real chain-correctness redirects to their manager
Duplicate-approver skip	same person resolved twice at one level → second assignment skipped → level still completes
Insufficient budget	submission blocked atomically — zero instances, assignments, or encumbrances created
Mixed split + unsplit report	both tracks (shared line item + owner's own split) must independently complete

5 tests new integration class 875 total

full suite, 1 pre-existing unrelated environment failure only

Reference — Every Endpoint Touched

METHOD	PATH	STATUS
--------	------	--------

GET / PUT	/xms/employee/expense-line-items/{lineItemId}/splits	New (Phase 2)
POST	/xms/approvals/{reportId}/splits/{splitId}/review	New (Phase 4)
GET	/xms/approvals/{reportId}/split-reviews	New (Phase 7)
POST	/xms/approvals/{reportId}/submit	Behavior extended (Phase 4/5) — same path
POST	/xms/approvals/{reportId}/bulk-approve	Behavior extended (Phase 7) — same path
POST	/xms/approvals/{reportId}/recall · /cancel · /reject	Behavior extended (Phase 5) — same paths
GET	/xms/approvals/my-queue	Response shape extended (Phase 7) — same path
POST / PUT / GET / DELETE	/xms/admin/cost-center-budgets[/ {budgetId}]	Request/response shape extended (Phase 3) — same paths

Reference — Every Entity Touched

ENTITY	STATUS	CHANGE
ExpenseSplit	NEW	one Cost Center allocation within a split line item
ApprovalSplitReview	NEW	per-split review, keyed by (split, assignment)
BudgetEncumbrance	NEW	ACTIVE/RELEASED/CONSUMED soft-reserve record
ExpenseLineItem	MODIFIED	+ expenseSplits collection
ApprovalAssignment	MODIFIED	+ splitReviews collection
CostCenter	MODIFIED	+ allowUnbudgeted
CostCenterBudget	MODIFIED	+ rolloverFromPrevious, allowRollover, rolloverCap, warningThreshold

Bugs Found & Fixed Along the Way

#	PHASE FOUND	WHAT IT WAS
1	6	Splits/encumbrances used native display currency instead of Org Base Currency — invisible unless a line item is priced in a foreign currency
2	7	A split-owner's own assignment could be wrongly marked complete by approving an unrelated shared line item

Known, Accepted Limitations

resumeInPlace staleness a same-cycle Manager correction never re-validates the encumbrance amount against a corrected total — accepted, documented (Phase 5)

Dual-role queue display if one person is both the header owner and a split owner at the same level, "My Queue" can only surface one of the two roles in a single row (Phase 7)

CostAllocation the legacy allocation entity remains in place, unmodified, pending a separate frontend-cutover decision (Phase 0)

Everything above is verified with **875 automated tests passing** across the full suite (one pre-existing, unrelated environmental failure — a test-only UMS_ISSUER_URI placeholder — present before this work began and untouched by it). **19 existing files modified, 27 new files added**, spanning entities, repositories, mappers, services, controllers, DTOs, and one database migration.

CONTENTS

- 00** Architecture mapping
- 01** Data foundations
- 02** Split validation
- 03** Budget encumbrance
- 04** Approval routing
- 05** Correction & lifecycle
- 06** AP completion
- 07** Queue & bulk approve
- 08** Integration testing

REFERENCE

- All endpoints
- All entities
- Bugs found & fixed
- Known limitations

